

SEMILLERO DE INVESTIGACIÓN SISINFO

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA — UNIPAZ

DOCUMENTACIÓN TÉCNICA Y DE ARQUITECTURA DE SOFTWARE

Especificación para Evaluadores de Código, Arquitectos y Auditoría de Seguridad (Versión 1.0)

Documento Oficial de Evaluación de Software — Repositorio GitHub: danielpose1996-stack/ingsisinfo

FICHA TÉCNICA DEL REPOSITORIO GITHUB

Parámetro Técnico	Detalle de Configuración
Repositorio de GitHub	danielpose1996-stack/ingsisinfo
Rama Principal	main
Framework Frontend	React 19.2.4 + Vite 6.1.1
Estilos & CSS	Tailwind CSS v4 + PostCSS + Framer Motion
Base de Datos & Backend	Supabase PostgreSQL + Row Level Security (RLS)
Linter & Calidad	ESLint 9 con reglas de React Hooks y Refresh

CAPÍTULO 1: ARQUITECTURA GLOBAL DEL SISTEMA

SISINFO adopta una arquitectura de Aplicación de Página Única (SPA - Single Page Application) basada en React 19, desacoplada de un Backend como Servicio (BaaS) operado sobre Supabase.

El flujo de datos sigue el patrón de unidireccionalidad de React con persistencia reactiva en tiempo real:

Flujo de Datos y Capa de Autenticación

```
[ Navegador Cliente / React 19 ]
  %
  %% (Estado Global & Tokens JWT)
[ AuthContext.jsx ] % % %° (Validación de Roles y Permisos)
  %
  %% (Abstracción de Servicios API)
[ src/lib/supabase.js ]
  %
  %% (Consultas Parametrizadas HTTPS / PostgREST)
[ Supabase Engine ]
  % % % %° PostgreSQL Database (Políticas RLS en Tablas)
  % % % %° Storage Buckets (documentos-proyectos & archivos-ova)
  % % % %° Edge Functions (create-user con service_role)
```

CAPÍTULO 2: MODELO RELACIONAL DE BASE DE DATOS Y POLÍTICAS RLS

La seguridad y aislamiento de los datos se implementa directamente en la capa de persistencia mediante Políticas de Seguridad a Nivel de Fila (RLS) en PostgreSQL.

2.1 Tablas del Sistema y Esquema Relacional

Nombre de Tabla	Clave Primaria / Foránea	Propósito en el Sistema
public.perfiles	id (PK -> auth.users.id)	Información de perfil, nombre, rol y carrera/línea.
public.proyectos	id (PK UUID), estudiante_id, docente_id	Propuestas de investigación y estado de avance.
public.versiones_proyecto	id (PK UUID), proyecto_id (FK)	Historial de entregables .docx subidos.
public.observaciones	id (PK UUID), proyecto_id (FK)	Retroalimentaciones emitidas por los docentes.
public.notificaciones	id (PK UUID), usuario_id (FK)	Alertas del sistema enviadas a usuarios.
public.modulos	id (PK UUID)	Módulos educativos del Aula Virtual.
public.ovas	id (PK UUID), modulo_id (FK)	Objetos Virtuales de Aprendizaje con contenido .JSONB.
public.seguimiento_ovas	id (PK UUID), ova_id (FK), usuario_id	Registro de resultados de quizzes presentados.

2.2 Definición de Políticas RLS (Security Policies)

A continuación se resumen las políticas SQL configuradas en la migración `supabase/migrations/secure\_rol\_policies.sql`:

Extracto de Políticas RLS en PostgreSQL

```
-- Lectura pública de perfiles autenticados
CREATE POLICY "Permitir lectura de perfiles a usuarios autenticados"
ON public.perfiles FOR SELECT
TO authenticated USING (true);

-- Restricción estricta de actualización de perfiles (Evita escalamiento de rol)
CREATE POLICY "Permitir actualización de perfil propio sin cambiar rol"
ON public.perfiles FOR UPDATE
TO authenticated
USING (auth.uid() = user_id OR auth.uid() = id)
WITH CHECK (
  (auth.uid() = user_id OR auth.uid() = id) AND
  rol = (SELECT rol FROM public.perfiles WHERE id = auth.uid())
);
```

PROTECCIÓN CONTRA ESCALAMIENTO DE PRIVILEGIOS

El trigger de la base de datos e instrucciones WITH CHECK impiden que un estudiante o docente modifique su propio campo "rol" a "admin" enviando cargas maliciosas desde la consola del navegador.

CAPÍTULO 3: ESTRUCTURA DE DIRECTORIOS Y MÓDULOS DE CÓDIGO

El repositorio se organiza con una división clara de responsabilidades:

Ruta de Archivo	Tipo de Componente	Responsabilidad Técnica
src/App.jsx	Router Principal	Definición de rutas públicas y protegidas con ProtectedRoute.
src/context/AuthContext.jsx	Contexto Global	Gestión de sesión Supabase Auth, perfil activo y permisos isAdmin.
src/lib/supabase.js	Capa de Datos	Encapsulamiento de clientes Supabase, funciones CRUD y Storage.
src/lib/security.js	Seguridad	Sanitización de cadenas HTML/texto mediante DOMPurify.
src/components/RichTextEditor.jsx	Editor UI	Integración TipTap con soporte de tablas e imágenes redimensionables.
src/components/QuizBuilder.jsx	Constructor UI	Editor interactivo de quizzes multimodales con dnd-kit.
src/components/QuizPlayer.jsx	Reproductor UI	Ejecución y calificación automatizada de quizzes para estudiantes.

CAPÍTULO 4: INTEGRACIÓN CON SUPABASE STORAGE Y EDGE FUNCTIONS

4.1 Almacenamiento de Archivos (Buckets)

📁 Documentos-proyectos: Bucket privado de almacenamiento para archivos de propuestas y entregas finales (.docx). Los enlaces de descarga se generan mediante URLs firmadas con vencimiento temporal para garantizar la privacidad.

📁 Archivos-ova: Bucket público optimizado para imágenes de portadas, recursos didácticos y contenido estático de los OVAs.

4.2 Edge Function `create-user`

La creación de usuarios con roles elevados o provisión administrativa se realiza a través de la Edge Function desplegada en Deno:

Estructura de la Edge Function create-user

```
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2'

Deno.serve(async (req) => {
  const supabaseAdmin = createClient(
    Deno.env.get('SUPABASE_URL'),
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') // Clave privada aislada en el servidor
  );
  // 1. Crea usuario en auth.users
  // 2. Inserta registro con rol verificado en public.perfiles
});
```

CAPÍTULO 5: AUDITORÍA DE SEGURIDAD Y CUMPLIMIENTO OWASP

Se ha verificado la conformidad del proyecto frente a los estándares de seguridad web OWASP:

- 🛡️ Prevención XSS:** Sanitización estricta de todo HTML renderizado en OVAs y vistas mediante DOMPurify.
- 🛡️ Prevención Inyección SQL:** Todas las consultas a PostgreSQL se ejecutan mediante peticiones HTTP parametrizadas gestionadas por la librería oficial `@supabase/supabase-js`.
- 🔑 Verificación de Secreto Cero:** Se audió el código completo confirmando que ninguna clave maestra (`SUPABASE\_SERVICE\_ROLE\_KEY`) ni contraseña real permanezca expuesta en el repositorio rastreado por Git.

CAPÍTULO 6: GUÍA DE COMPILACIÓN Y DESPLIEGUE EN PRODUCCIÓN

Pasos para compilar y desplegar la aplicación en cualquier servidor o plataforma Cloud (Vercel, Netlify):

Paso 1. Instalación de Dependencias — `npm install`

Paso 2. Compilación de Producción — `npm run build` (Genera bundle optimizado en directorio `/dist`).

Paso 3. Configuración Vercel — El archivo `vercel.json` incluye la regla de reescritura `{"source": "/*.*", "destination": "/index.html"}` para dar soporte completo a las rutas del cliente.

